

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 16 : Créer et Modifier des Bibliothèques

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

16.0 Introduction : Pourquoi créer une bibliothèque ?

De l'Artisanat à l'Ingénierie Logicielle

Lorsque vos programmes grandissent, copier-coller les mêmes blocs de code d'un projet à l'autre devient source d'erreurs. Une bibliothèque (Library) permet d'encapsuler des fonctionnalités complexes dans une "boîte noire" réutilisable.

Dans ce chapitre, nous allons apprendre à :

- Comprendre l'architecture d'une bibliothèque Arduino.
- Écrire un fichier d'en-tête (`.h`) et un fichier source (`.cpp`).
- Introduire les concepts de Programmation Orientée Objet (Classes, Objets, Méthodes).
- Utiliser `Arduino.h` pour accéder aux fonctions natives.
- Coloriser la syntaxe avec `keywords.txt` et ajouter des exemples.

16.1 L'Anatomie d'une Bibliothèque Arduino

Une bibliothèque n'est pas un seul fichier, c'est un **dossier** placé dans Documents/Arduino/libraries/.

Structure standard du dossier (ex : MonCapteur) :

- `MonCapteur.h` : Le fichier d'en-tête (Header). Il déclare "ce que fait" la bibliothèque (le plan de la maison).
- `MonCapteur.cpp` : Le fichier source (C++). Il contient le code réel (la construction de la maison).
- `keywords.txt` (Optionnel) : Indique à l'IDE Arduino quels mots mettre en couleur (orange ou bleu).
- `examples/` (Optionnel) : Un sous-dossier contenant des croquis (`.ino`) pour montrer comment utiliser la bibliothèque.

16.2 Rappel C++ : La Programmation Orientée Objet (POO)

Le langage Arduino est du C/C++. Pour écrire une bibliothèque, on utilise le concept de **Classe**.

- **La Classe (Class)** : C'est un moule, un concept abstrait. Par exemple, la classe `Voiture` possède des attributs (couleur, vitesse) et des méthodes (accélérer, freiner).
- **L'Objet (Object)** : C'est la concrétisation de la classe. Par exemple, `maPeugeot` est un objet de la classe `Voiture`, de couleur rouge.

*En Arduino, quand vous tapez `Servo monMoteur;`, vous créez un **Objet** nommé `monMoteur` basé sur la **Classe** `Servo`.*

16.3 Les droits d'accès : Public vs Private

Une classe protège ses données internes pour éviter que l'utilisateur du code ne fasse des erreurs (Encapsulation).

- **public**: Ce qui est placé ici peut être utilisé par le programmeur dans son croquis Arduino (ex : la méthode `monMoteur.write(90);`).
- **private**: Ce qui est placé ici est strictement réservé à l'usage interne de la classe. Le programmeur n'y a pas accès directement (ex : la broche matérielle exacte mémorisée par le moteur).

16.4 Le Fichier d'En-tête (.h)

Objectif : Créer une bibliothèque `Clignotant` pour faire clignoter une LED sans bloquer le programme (multitâche vu au Chapitre 12).

Les "Include Guards"

Un fichier `.h` doit toujours commencer par des directives de préprocesseur (`#ifndef`, `#define`) pour éviter qu'il ne soit inclus deux fois par erreur lors de la compilation.

Code : Le fichier Clignotant.h

```
// --- Fichier : Clignotant.h ---
#ifndef Clignotant_h
#define Clignotant_h

#include "Arduino.h" // Indispensable pour utiliser digitalWrite, millis, etc.

class Clignotant {
public:
    // Le Constructeur : Appele lors de la creation de l'objet
    Clignotant(int broche, long intervalle);

    // La methode principale a appeler dans loop()
    void actualiser();

private:
    int _broche;           // Par convention, les variables privees commencent par '_'
    long _intervalle;
    unsigned long _tempsPrecedent;
    int _etatLed;
};

#endif
```

16.5 Le Fichier Source (.cpp)

C'est ici que l'on écrit la logique mathématique et matérielle.

L'opérateur de résolution de portée (::) : Dans le fichier .cpp, on doit expliquer au compilateur que la fonction `actualiser()` que l'on écrit appartient bien à la classe `Clignotant`, et n'est pas une simple fonction perdue.

On l'écrit donc : `void Clignotant::actualiser() { ... }`

Code : Le fichier Clignotant.cpp (Constructeur)

```
// --- Fichier : Clignotant.cpp ---  
#include "Arduino.h"  
#include "Clignotant.h"  
  
// Definition du Constructeur  
// Il initialise les variables privees et configure la broche  
Clignotant::Clignotant(int broche, long intervalle) {  
    _broche = broche;  
    _intervalle = intervalle;  
    _etatLed = LOW;  
    _tempsPrecedent = 0;  
  
    pinMode(_broche, OUTPUT); // Configure le materiel  
}
```

Code : Le fichier Clignotant.cpp (La méthode)

Voici l'implémentation du multitâche "Blink Without Delay" caché à l'intérieur de notre bibliothèque.

```
// Definition de la methode d'actualisation
void Clignotant::actualiser() {
    unsigned long tempsActuel = millis();

    if (tempsActuel - _tempsPrecedent >= _intervalle) {
        _tempsPrecedent = tempsActuel;

        if (_etatLed == LOW) {
            _etatLed = HIGH;
        } else {
            _etatLed = LOW;
        }
        digitalWrite(_broche, _etatLed);
    }
}
```

16.6 Utiliser notre nouvelle Bibliothèque

Maintenant que la bibliothèque est placée dans le dossier `libraries`, voici comment l'utilisateur final va s'en servir dans l'IDE Arduino.

```
#include <Clignotant.h>

// On cree deux Objets independants !
Clignotant ledRouge(12, 1000); // Broche 12, clignote toutes les secondes
Clignotant ledVerte(13, 250);  // Broche 13, clignote tres vite (250ms)

void setup() {
    // Rien a faire ici, le constructeur s'en est deja occupe !
}

void loop() {
    // On appelle les methodes en boucle
    ledRouge.actualiser();
    ledVerte.actualiser();
}
```

16.7 Le fichier `keywords.txt` (Esthétique)

Pour que vos utilisateurs aient une belle coloration syntaxique dans l'IDE Arduino, créez un simple fichier texte nommé `keywords.txt` dans le dossier de la bibliothèque.

Format strict : Le nom, suivi d'une **Tabulation** (pas d'espaces!), suivi du code couleur (KEYWORD1 pour les classes, KEYWORD2 pour les méthodes).

Contenu de `keywords.txt`

<code>Clignotant</code>	KEYWORD1
<code>actualiser</code>	KEYWORD2

16.8 Le dossier `examples/`

Une bibliothèque sans documentation ne sert à rien. L'IDE Arduino cherche automatiquement un dossier nommé `examples` dans votre bibliothèque.

- Créez un dossier `examples` à la racine de votre bibliothèque.
- À l'intérieur, créez un dossier `SimpleClignotement`.
- Mettez-y votre fichier `SimpleClignotement.ino`.
- Redémarrez l'IDE.
- **Résultat** : Votre croquis apparaîtra dans le menu *Fichier > Exemples > Clignotant > SimpleClignotement*.

16.9 Le Piège : `#include "Arduino.h"`

L'erreur du débutant : Lorsqu'on écrit un croquis standard (`.ino`), l'IDE Arduino ajoute secrètement les bibliothèques de base.

Mais dans un fichier `.cpp` ou `.h` personnalisé, le compilateur C++ est aveugle ! Il ne connaît ni `digitalWrite`, ni `HIGH`, ni `byte`, ni `millis()`.

La Règle d'or : Il faut **absolument** placer `#include "Arduino.h"` au début de tous vos fichiers `.h` et `.cpp` pour lier votre code à l'écosystème matériel de la carte.

16.10 Modifier une bibliothèque existante

Parfois, une bibliothèque téléchargée sur internet (ex : via le Library Manager) a un petit bug ou manque d'une fonctionnalité.

- Les bibliothèques téléchargées se trouvent dans `Documents/Arduino/libraries` (Windows/Mac) ou `~/Arduino/libraries` (Linux).
- Vous pouvez ouvrir directement les fichiers `.cpp` avec un éditeur de texte (Notepad++, VS Code).
- **Attention** : Si l'auteur met à jour la bibliothèque via le gestionnaire de l'IDE, vos modifications locales seront écrasées ! Il vaut mieux renommer la bibliothèque modifiée.

Résumé du Chapitre 16

- **Modularité** : Créer des bibliothèques permet de garder un fichier `.ino` propre et lisible (souvent réduit à moins de 20 lignes).
- **La paire `.h` / `.cpp`** : Le `.h` déclare l'interface et les variables, le `.cpp` contient le moteur logique du code.
- **Encapsulation (Public/Private)** : Protégez les variables internes (broches matérielles, chronomètres) dans la section `private`: pour éviter qu'un utilisateur maladroit ne les écrase accidentellement.
- **Partage** : Une fois structurée avec un fichier de mots-clés et des exemples, vous pouvez compresser votre dossier en `.zip` et le partager avec la communauté Arduino du monde entier !

Fin du Chapitre 16. Souhaitez-vous aborder le Chapitre 17, consacré au Code Avancé et à la Gestion pointue de la Mémoire (Advanced Coding and Memory Handling) ?